LAPORAN PRAKTIKUM

STRUKTUR DATA

ABSTRACT DATA TYPE JAM

Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan ke-1 dengan judul *"Implementasi Abstract Data Type (ADT) dalam Pemrograman Java"* tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep Abstract Data Type (ADT) serta penerapannya dalam pemrograman Java. Melalui praktikum ini, penulis mempelajari bagaimana merepresentasikan data waktu (jam) ke dalam sebuah objek serta mengimplementasikan berbagai operasi, seperti validasi waktu, konversi ke dalam satuan detik, operasi aritmatika (penjumlahan dan pengurangan), serta operasi relasional (perbandingan antar waktu).

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Ucapan terima kasih penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang telah membantu dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 1 April 2026

Penulis

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam pengembangan pemrograman, pengolahan data yang terstruktur sangat penting untuk menghasilkan program yang efisien, terorganisir, dan mudah dikembangkan. Salah satu konsep yang digunakan dalam struktur data adalah *Abstract Data Type (ADT)*, yaitu suatu tipe data abstrak yang mendefinisikan data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan implementasi internalnya.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara merancang dan mengimplementasikan *Abstract Data Type (ADT)* menggunakan bahasa pemrograman Java, mulai dari pembuatan tipe data objek (*class*), penentuan atribut seperti jam, menit, dan detik, hingga implementasi berbagai operasi seperti validasi, konversi ke dalam satuan detik, operasi aritmatika, serta operasi relasional untuk mengatur interaksi dan pengolahan data waktu dalam program.

1.2. Tujuan

Tujuan praktikum:

1.Untuk memahami konsep dasar *Abstract Data Type (ADT)* dalam struktur data.
2.Untuk mengimplementasikan *Abstract Data Type (ADT)* dalam bahasa pemrograman
3.Untuk membuat ADT Jam beserta operasi-operasinya, meliputi validasi, konversi, operasi aritmatika, dan operasi relasional.

1.3. Manfaat

Manfaat yang diharapkan dari praktikum:

1.Mahasiswa dapat memahami konsep *Abstract Data Type (ADT)* secara lebih mendalam melalui penerapan langsung.
2.Mahasiswa dapat meningkatkan kemampuan dalam merancang dan mengembangkan program menggunakan bahasa pemrograman Java.
3.Mahasiswa dapat melatih kemampuan dalam mengimplementasikan operasi-operasi pada suatu tipe data, seperti validasi, konversi, aritmatika, dan relasional.
4.Mahasiswa dapat meningkatkan kemampuan berpikir logis dan terstruktur dalam menyelesaikan permasalahan pemrograman.

BAB II

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah *Abstract Data Type (ADT)*. ADT adalah suatu tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana implementasinya secara detail.

Konsep utama dari ADT adalah pemisahan antara *interface* (apa yang dapat dilakukan) dan *implementasi* (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer dapat fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail internalnya. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan *class*. Sebuah *class* berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diimplementasikan adalah ADT Jam. ADT ini digunakan untuk merepresentasikan data waktu yang terdiri dari jam, menit, dan detik. Setiap atribut memiliki batasan nilai tertentu, yaitu jam (0–23), menit (0–59), dan detik (0–59). Oleh karena itu, diperlukan proses validasi untuk memastikan bahwa data yang dimasukkan sesuai dengan batasan tersebut.

Selain itu, ADT Jam juga dilengkapi dengan beberapa operasi, antara lain:

1. Validasi waktu, untuk memastikan nilai jam, menit, dan detik berada dalam rentang yang benar.
2. Konversi ke detik, yaitu mengubah waktu ke dalam satuan detik untuk memudahkan perhitungan.
3. Operasi aritmatika, seperti penjumlahan dan pengurangan waktu.
4. Operasi relasional, yaitu membandingkan dua waktu untuk mengetahui apakah suatu waktu lebih besar, lebih kecil, atau sama dengan waktu lainnya.

Dengan memahami dan mengimplementasikan ADT Jam, mahasiswa dapat mengembangkan kemampuan dalam merancang struktur data yang baik serta mengelola data secara lebih sistematis dalam pemrograman.

1.Langkah Kerja

1) Jam_2511531014

- **Kode Program**

```java
package pekan1_2511531014;

package pekan1_2511531014;

public class jam_2511531014 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    //--Validator--
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    // -- Kontruktor --
    public jam_2511531014(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    //--Selektor--
    public int getHour() {return hh; }
    public int getMinute() {return mm; }
    public int getSecond() {return ss; }

    //--Mutator--
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m; }
    public void setSecond(int s) {this.ss = s; }


//--Konversi--
public int toSecond() {return hh * 3600 + mm * 60 + ss; }
public static jam_2511531014 fromSecond (int total ) {
    if (total < 0) throw new IllegalArgumentException ("detik negatif");
    total %= 24 * 3600; //wrap 24 jam
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new jam_2511531014(h, m, s);
}

//--relasional--
public int compareTo(jam_2511531014 other) { return Integer.compare(this. toSecond(), other.toSecond()); }
public boolean equals(Object o) {
    if (!(o instanceof jam_2511531014 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int hashCode() {return java.util.Objects.hash(hh, mm, ss); }

// --- Aritmatika --
public jam_2511531014 plus(jam_2511531014 other) {return fromSecond(this.toSecond() + other.toSecond()); }

public jam_2511531014  minus(jam_2511531014  other) {return fromSecond(Math.floorMod(this.toSecond() - other.toSecond(), 24 * 3600)); }

public jam_2511531014 nextSecond() { return fromSecond(this.toSecond() + 1); }

public jam_2511531014 nextNSeconds(int n) { return fromSecond(this.toSecond() + Math.max(0, n)); }

public jam_2511531014  prevSecond() {return fromSecond(Math.floorMod(this.toSecond() - 1, 24 * 3600)); }

public jam_2511531014  prevNSeconds(int n) {return fromSecond(Math.floorMod(this.toSecond() - Math.max(0, n), 24 * 3600)); }

// --- Durasi (detik) --- bisa negatif jika this > other sesuai spesifikasi durasi (JAw, JAk)
public static int durasiDetik(jam_2511531014 jaw, jam_2511531014 jakh) {return jakh.toSecond() - jaw.toSecond(); }
// overriding ke string
public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }


}
```

- **Pembahasan**

1. Program ini dibuat dengan mendefinisikan sebuah *class* bernama Jam_2511531018 yang digunakan untuk merepresentasikan waktu. Pada *class* ini terdapat tiga atribut utama, yaitu hh yang menyatakan jam dengan rentang nilai 0–23, mm yang menyatakan menit dengan rentang nilai 0–59, serta ss yang menyatakan detik dengan rentang nilai 0–59.

```java
public class jam_2511531014 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59
```

2. Method ini digunakan untuk mengecek apakah input waktu valid. Mengembalikan true jika jam antara 0-23 dan menit & detik antara 0-59. Kalau tidak sesuai maka false.

```java
//--Validator--
public static boolean isValid(int h, int m, int s) {
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
}
```

3. Digunakan saat membuat objek jam. Nilai h, m, s langsung disimpan ke atribut.

```java
// -- Kontruktor --
public jam_2511531014(int h, int m, int s) {
    this.hh = h; this.mm = m; this.ss = s;
}
```

4. Mengambil nilai jam, menit, dan detik dari objek.

```java
//--Selektor--
public int getHour() {return hh; }
public int getMinute() {return mm; }
public int getSecond() {return ss; }
```

5. Mengubah nilai jam, menit, dan detik dari objek.

```java
//--Mutator--
public void setHour(int h) {this.hh = h; }
public void setMinute(int m) {this.mm = m; }
public void setSecond(int s) {this.ss = s; }
```

6. Mengubah waktu menjadi total detik dengan rumus jam * 3600, menit * 60, dan detik langsung

```java
//--Konversi--
public int toSecond() {return hh * 3600 + mm * 60 + ss; }
```

7. Jika detik negatif maka error, jika lebih dari 24 jam maka diputar (modulo 24 jam). Konversi jam = total/3600, sisa => menit, dan sisa => detik. Buat objek bari dari hasil tersebut.

```java
public static jam_2511531014 fromSecond (int total ) {
```

8. Membandingkan dua waktu berdasarkan total detik. Hasil negatif => lebih kecil, 0 => sama, dan positif => lebih besar. Mengecek apakah dua objek waktu sama persis.

```java
//--relasional--
public int compareTo(jam_2511531014 other) { return Integer.compare(this. toSecond(), other.toSecond()); }
public boolean equals(Object o) {
    if (!(o instanceof jam_2511531014 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int hashCode() {return java.util.Objects.hash(hh, mm, ss); }
```

9. Menjumlahkan dua waktu, mengurangkan waktu, memakai floormod agar tidak negatif dan menjadi 24 jam, menambah detik, dan mengurangi detik.

```java
// --- Aritmatika --
public jam_2511531014 plus(jam_2511531014 other) {return fromSecond(this.toSecond() + other.toSecond()); }

public jam_2511531014  minus(jam_2511531014  other) {return fromSecond(Math.floorMod(this.toSecond() - other.toSecond(), 24 * 3600)); }

public jam_2511531014 nextSecond() { return fromSecond(this.toSecond() + 1); }

public jam_2511531014 nextNSeconds(int n) { return fromSecond(this.toSecond() + Math.max(0, n)); }

public jam_2511531014  prevSecond() {return fromSecond(Math.floorMod(this.toSecond() - 1, 24 * 3600)); }

public jam_2511531014  prevNSeconds(int n) {return fromSecond(Math.floorMod(this.toSecond() - Math.max(0, n), 24 * 3600)); }
```

10. Menghitung selisih waktu dalam detik: waktu akhir – waktu awal.

```java
// --- Durasi (detik) --- bisa negatif jika this > other sesuai spesifikasi durasi (JAw, JAk)
public static int durasiDetik(jam_2511531014 jaw, jam_2511531014 jakh) {return jakh.toSecond() - jaw.toSecond(); }
```

11. Mengubah waktu menjadi format dan selalu 2 digit agar rapi.

```java
// overriding ke string
public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

2) JamDriver_2511531014

- **Kode Program**

```java
package pekan1_2511531014;

public class JamDriver_2511531014 {
    public static void main (String [] args) {
        jam_2511531014 a = new jam_2511531014(23, 59, 50);
        jam_2511531014 b = new jam_2511531014(0, 0, 15);
        System.out.println("a                 =" + a);
        System.out.println("b                 =" + b);
        System.out.println("a+b               =" + a.plus(b));
        System.out.println("next 20s          =" + a.nextNSeconds(20));
        System.out.println("durasi (a,b)      =" + jam_2511531014.durasiDetik(a, b));
        System.out.println("a.compareTo(b)    =" + a.compareTo (b));

    }
```

- **Pembahasan**

1. Program utama dimulai dari *method* main, di mana seluruh objek dibuat dan dijalankan.

```
public class JamDriver_2511531014 {
    public static void main (String [] args) {
```

2. Membuat dua objek waktu, yaitu objek **a** dengan nilai waktu 23:59:50 dan objek **b** dengan nilai waktu 00:00:15. Hal ini menunjukkan bahwa waktu **a** berada mendekati tengah malam, sedangkan waktu **b** sudah melewati tengah malam.

```
jam_2511531014 a = new jam_2511531014(23, 59, 50);
jam_2511531014 b = new jam_2511531014(0, 0, 15);
```

3. Memanggil *method* toString() untuk menampilkan nilai waktu dari objek **a** dan **b**.

```
System.out.println("a                    =" + a);
System.out.println("b                    =" + b);
```

4. Mengubah waktu **a** dan **b** ke dalam satuan detik, kemudian menjumlahkannya, dan hasilnya dikonversi kembali ke dalam format jam, menit, dan detik.

```
System.out.println("a+b                  =" + a.plus(b));
```

5. Menambahkan 20 detik ke waktu **a** untuk melihat perubahan waktu setelah dilakukan operasi aritmatika.

```
System.out.println("next 20s          =" + a.nextNSeconds(20));
```

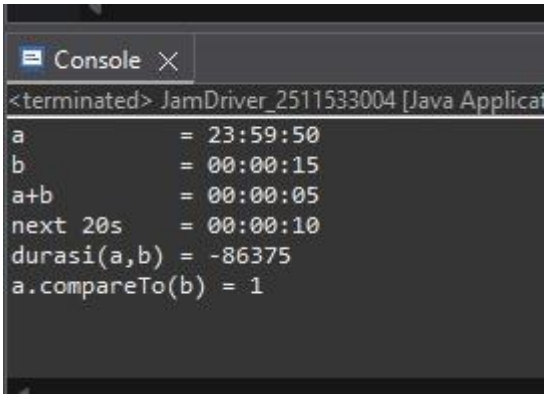
6. Menghitung durasi antara waktu **a** dan **b** menggunakan *method* durasiDetik(a, b), yaitu untuk mengetahui selisih waktu dalam satuan detik.

```
System.out.println("durasi (a,b)     =" + jam_2511531014.durasiDetik(a, b));
```

7. Menggunakan *method* compareTo() untuk membandingkan waktu **a** dan **b** berdasarkan nilai detiknya, sehingga dapat diketahui apakah waktu **a** lebih besar, lebih kecil, atau sama dengan waktu **b**.

```
System.out.println("durasi (a,b)     =" + jam_2511531014.durasiDetik(a, b));
```

8. Output sebagai berikut



3) **JamDriver2_2511531014**

- **Kode Program**

```java
package pekan1_2511531014;

    import java.util.Scanner;

    public class JamDriver2_2511531014 {
        public static void main(String[] args) {
            Scanner input = new Scanner(System.in);

            System.out.println("\n=== Program Driver Objek Jam ===");

            // 1. Input Jam Pertama
            System.out.println("\n--- Input jam_2511531014 1 ---");
            jam_2511531014 j1 = buatjam_2511531014DariInput(input);

            // 2. Input Jam Kedua
            System.out.println("\n--- Input jam_2511531014 2 ---");
            jam_2511531014 j2 = buatjam_2511531014DariInput(input);

            // 3. Menampilkan Data
            System.out.println("\n--- Hasil Operasi ---");
            System.out.println(" jam_2511531014 1 (String)       : " + j1.toString());
            System.out.println(" jam_2511531014 2 (String)       : " + j2.toString());
            System.out.println(" jam_2511531014 1 dalam detik    : " + j1.toSecond());
            System.out.println(" jam_2511531014 2 dalam detik    : " + j2.toSecond());

            //4. O[erasi relasional (perbandingan)
            int perbandingan = j1.compareTo (j2);
            if (perbandingan > 0) {
                System.out.println("Status    : jam_2511531014 1 lebih lambat (setelah) jam_2511531014 2");
            } else if (perbandingan < 0) {
                System.out.println("Status    : jam_2511531014 1 lebih awal (sebelum) jam_2511531014 2");
            } else {
                System.out.println("Status    : jam_2511531014 1 dan) jam_2511531014 2 sama persis");
            }
```

```java
        //5.Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)  :" +jam_2511531014.durasiDetik(j1, j2) + "detik");

        jam_2511531014 jNext = j1.nextSecond();
        System.out.println("jam_2511531014 1 Detik Berikutnya:" + jNext);

        jam_2511531014 jPrev = j1.prevSecond();
        System.out.println("jam_2511531014 1 Detik Sebelumnya:" + jPrev);

       //6.Operasi Penjumlahan jam_2511531014
        jam_2511531014 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2      :" + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }

    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */
    private static jam_2511531014 buatjam_2511531014DariInput(Scanner sc) {
        int h, m, s;

        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

              // Memanggil method static isValid dari kelas jam_2511531014
              if (jam_2511531014.isValid(h, m, s)) {
                  return new jam_2511531014(h, m, s);
              } else {
                  System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
              }
          }
      }
}
```

- **Pembahasan**

    1. Program dimulai dari *method* main, kemudian dibuat objek Scanner untuk menerima input dari pengguna, serta menampilkan judul program.

```java
public class JamDriver2_2511531014 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```

```java
System.out.println("\n=== Program Driver Objek Jam ===");
```

2. Menampilkan label input, kemudian memanggil fungsi buatJamDariInput untuk menerima data dari pengguna dan menghasilkan dua objek waktu, yaitu **j1** dan **j2**.

```java
// 1. Input Jam Pertama
System.out.println("\n--- Input jam_2511531014 1 ---");
jam_2511531014 j1 = buatjam_2511531014DariInput(input);

// 2. Input Jam Kedua
System.out.println("\n--- Input jam_2511531014 2 ---");
jam_2511531014 j2 = buatjam_2511531014DariInput(input);
```

3. Menggunakan *method* toString() untuk menampilkan format waktu, serta *method* toSeconds() untuk mengonversi waktu ke dalam total detik.

```java
// 3. Menampilkan Data
System.out.println("\n--- Hasil Operasi ---");
System.out.println(" jam_2511531014 1 (String)       : " + j1.toString());
System.out.println(" jam_2511531014 2 (String)       : " + j2.toString());
System.out.println(" jam_2511531014 1 dalam detik    : " + j1.toSecond());
System.out.println(" jam_2511531014 2 dalam detik    : " + j2.toSecond());
```

4. Membandingkan kedua waktu berdasarkan nilai detiknya untuk mengetahui hubungan antara **j1** dan **j2** (lebih besar, lebih kecil, atau sama).

```java
//4. Operasi relasional (perbandingan)
int perbandingan = j1.compareTo (j2);
if (perbandingan > 0) {
    System.out.println("Status    : jam_2511531014 1 lebih lambat (setelah) jam_2511531014 2");
} else if (perbandingan < 0) {
    System.out.println("Status    : jam_2511531014 1 lebih awal (sebelum) jam_2511531014 2");
} else {
    System.out.println("Status    : jam_2511531014 1 dan) jam_2511531014 2 sama persis");
}
```

5. Melakukan beberapa operasi terhadap waktu, yaitu menghitung selisih waktu dalam detik, menambahkan 1 detik pada **j1**, mengurangi 1 detik, serta menjumlahkan dua waktu.

```java
//5.Operasi Aritmatika
System.out.println("Durasi (J1 ke J2)  :" +jam_2511531014.durasiDetik(j1, j2) + "detik");

jam_2511531014 jNext = j1.nextSecond();
System.out.println("jam_2511531014 1 Detik Berikutnya:" + jNext);

jam_2511531014 jPrev = j1.prevSecond();
System.out.println("jam_2511531014 1 Detik Sebelumnya:" + jPrev);

//6.Operasi Penjumlahan jam_2511531014
jam_2511531014 jHasilPlus = j1.plus(j2);
System.out.println("Hasil J1 + J2      :" + jHasilPlus);
```

6. Menutup objek Scanner setelah selesai digunakan, kemudian menampilkan hasil akhir program.

```java
    input.close();
    System.out.println("\nProgram Selesai.");
}
```

7. Program menggunakan perulangan (*loop*) untuk terus meminta input dari pengguna sampai data yang dimasukkan valid.

```java
private static jam_2511531014 buatjam_2511531014DariInput(Scanner sc) {
    int h, m, s;

    while (true) {
        System.out.print("Masukkan Jam (0-23)   : ");
        h = sc.nextInt();
        System.out.print("Masukkan Menit (0-59) : ");
        m = sc.nextInt();
        System.out.print("Masukkan Detik (0-59) : ");
        s = sc.nextInt();
```

8. Dilakukan proses validasi terhadap input. Jika data yang dimasukkan tidak sesuai (di luar rentang jam, menit, atau detik), maka program akan kembali ke awal perulangan hingga input dinyatakan valid.

```java
// Memanggil method static isValid dari kelas jam_2511531014
if (jam_2511531014.isValid(h, m, s)) {
    return new jam_2511531014(h, m, s);
} else {
    System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
}
```

9. Output yang dihasilkan

```
Console
<terminated> JamDriver2_2511533004 [Java Application]
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)   : 22
Masukkan Menit (0-59) : 50
Masukkan Detik (0-59) : 55

--- Input Jam 2 ---
Masukkan Jam (0-23)   : 20
Masukkan Menit (0-59) : 58
Masukkan Detik (0-59) : 56

--- Hasil Operasi ---
Jam 1 (String)      : 22:50:55
Jam 2 (String)      : 20:58:56
Jam 1 dalam detik   : 82255
Jam 2 dalam detik   : 75536
Status : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2) : -6719 detik
Jam 1 Detik Berikutnya: 22:50:56
Jam 1 Detik Sebelumnya: 22:50:54
Hasil J1 + J2 : 19:49:51

Program Selesai.
```

KESIMPULAN

Berdasarkan program yang telah dibuat, dapat disimpulkan bahwa implementasi *class* **Jam_2511531014**, **JamDriver_2511531014**, dan **JamDriver2_2511531014** telah berhasil merepresentasikan serta mengelola data waktu dalam format jam, menit, dan detik. Program ini mampu melakukan berbagai operasi waktu, seperti validasi input, konversi ke dalam satuan detik, serta konversi kembali ke format waktu semula.

Selain itu, program juga mendukung operasi aritmatika waktu, seperti penjumlahan, pengurangan, serta penambahan detik dengan memperhatikan batasan 24 jam (*wrap around*). Fitur perbandingan waktu dan perhitungan durasi menunjukkan bahwa pengolahan waktu dapat dilakukan secara sistematis dengan menjadikan detik sebagai dasar perhitungan.

Dengan adanya program *driver*, seluruh fungsi dalam *class* telah diuji dan hasilnya dapat ditampilkan dengan baik, sehingga membuktikan bahwa program berjalan sesuai dengan logika yang telah dirancang. Secara keseluruhan, program ini memberikan pemahaman yang lebih baik mengenai konsep pemrograman berorientasi objek, khususnya dalam pengelolaan data waktu dan berbagai operasi yang berkaitan.

TUGAS PEKAN 1

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

1.Kelas ADTMobil_2511531014

- **Kode Program**

```java
package TugasPekan1_2511531014;

public class ADTMobil_2511531014 {

        private String nama;
        private int tahun;
        private int cc;
        private double harga;
        private String merk;

        // Constructor
        public ADTMobil_2511531014(String nama, int tahun, int cc, double harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }

        // ===== GETTER (SELEKTOR) =====
        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCc() { return cc; }
        public double getHarga() { return harga; }
        public String getMerk() { return merk; }

        // ===== SETTER (MUTATOR) =====
        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(double harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk; }

        // ===== METHOD TAMPIL =====
        public void tampil() {
            System.out.println(nama + " | " + tahun + " | " + cc + "cc | " + harga + " | " + merk);
        }

        // ===== ARRAY (STATIC) =====
        public static ADTMobil_2511531014[] data = new ADTMobil_2511531014[10];
        public static int n = 0;

        // ===== TAMBAH MOBIL =====
        public static void tambahMobil(ADTMobil_2511531014 m) {
            if (n < data.length) {
                data[n] = m;
                n++;
            } else {
                System.out.println("Data penuh!");
            }
        }

        // ===== HAPUS MOBIL =====
        public static void hapusMobil(int index) {
            if (index >= 0 && index < n) {
                for (int i = index; i < n - 1; i++) {
                    data[i] = data[i + 1];
                }
                n--;
            } else {
                System.out.println("Index tidak valid!");
            }
        }

        // ===== TAMPIL SEMUA =====
        public static void tampilSemua() {
            for (int i = 0; i < n; i++) {
                data[i].tampil();
            }
        }
    }
```

- **Pembahasan**

1. Kelas **ADTMobil_2511531014** digunakan sebagai representasi Abstract Data Type (ADT) untuk data mobil.

```java
public class ADTMobil_2511531014 {

        private String nama;
        private int tahun;
        private int cc;
        private double harga;
        private String merk;

    // ===== TAMPIL SEMUA =====
    public static void tampilSemua() {
        for (int i = 0; i < n; i++) {
            data[i].tampil();
        }
    }
}
```

2. Constructor

```java
    // Constructor
    public ADTMobil_2511531014(String nama, int tahun, int cc, double harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
```

3. Selektor (getter)

Method getter digunakan untuk mengambil nilai atribut dari objek mobil.

```java
    // ===== GETTER (SELEKTOR) =====
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public double getHarga() { return harga; }
    public String getMerk() { return merk; }
```

4. Mutator(Setter)

Method setter digunakan untuk mengubah nilai atribut.

```java
    // ===== SETTER (MUTATOR) =====
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(double harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }
```

5. Method Tampil ()

Method tampil() digunakan untuk menampilkan data satu objek mobil ke layar dalam bentuk ringkas.

```java
// ===== METHOD TAMPIL =====
public void tampil() {
    System.out.println(nama + " | " + tahun + " | " + cc + "cc | " + harga + " | " + merk);
}
```

6. Array

```java
// ===== ARRAY (STATIC) =====
public static ADTMobil_2511531014[] data = new ADTMobil_2511531014[10];
public static int n = 0;
```

7. Method Tambah mobil()

```java
// ===== TAMBAH MOBIL =====
public static void tambahMobil(ADTMobil_2511531014 m) {
    if (n < data.length) {
        data[n] = m;
        n++;
    } else {
        System.out.println("Data penuh!");
    }
}
```

8. Method hapus mobil()

```java
// ===== HAPUS MOBIL =====
public static void hapusMobil(int index) {
    if (index >= 0 && index < n) {
        for (int i = index; i < n - 1; i++) {
            data[i] = data[i + 1];
        }
        n--;
    } else {
        System.out.println("Index tidak valid!");
    }
}
```

9. Method tampilsemua()

```java
// ===== TAMPIL SEMUA =====
public static void tampilSemua() {
    for (int i = 0; i < n; i++) {
        data[i].tampil();
    }
}
}
```

2.Kelas Driver_2511531014

- **Kode Program**

```java
package TugasPekan1_2511531014;

public class Driver_2511531014 {
    public static void main(String[] args) {

        ADTMobil_2511531014.tambahMobil(
            new ADTMobil_2511531014("Avanza", 2022, 1300, 150000000, "Toyota")
        );

        ADTMobil_2511531014.tambahMobil(
            new ADTMobil_2511531014("Brio", 2024, 1200, 200000000, "Honda")
        );

        ADTMobil_2511531014.tambahMobil(
            new ADTMobil_2511531014("Rush", 2021, 1500, 540000000, "Toyota")
        );

        System.out.println("Data Mobil:");
        ADTMobil_2511531014.tampilSemua();
    }

}
```

- **Pembahasan**

Kelas Driver_2511531014 berfungsi sebagai Class main (Program utama), Pada kelas ini dilakukan:

- tambahMobil()
- Menampilkan data dengan tampilSemua()
- Menghapus data dengan hapusMobil()

Driver hanya sebagai pengendali, sedangkan semua operasi utama berada di kelas ADT.

```java
ADTMobil_2511531014.tambahMobil(
    new ADTMobil_2511531014("Avanza", 2022, 1300, 150000000, "Toyota")
);

ADTMobil_2511531014.tambahMobil(
    new ADTMobil_2511531014("Brio", 2024, 1200, 200000000, "Honda")
);

ADTMobil_2511531014.tambahMobil(
    new ADTMobil_2511531014("Rush", 2021, 1500, 540000000, "Toyota")
);
```

Digunakan untuk menambahkan data mobil.

```java
    );

    System.out.println("Data Mobil:");
    ADTMobil_2511531014.tampilSemua();
}
```

Digunakan untuk menmapilkan semua data. Dan akan menghasilkan Output sebagai berikut

```
Console
<terminated> Driver_2511531014 [Java Application] C:\Users\naysh\.p
Data Mobil:
Avanza | 2022 | 1300cc | 1.5E8 | Toyota
Brio | 2024 | 1200cc | 2.0E8 | Honda
Rush | 2021 | 1500cc | 5.4E8 | Toyota
```